CS 5588 Data Science Capstone — Challenge 1 Report

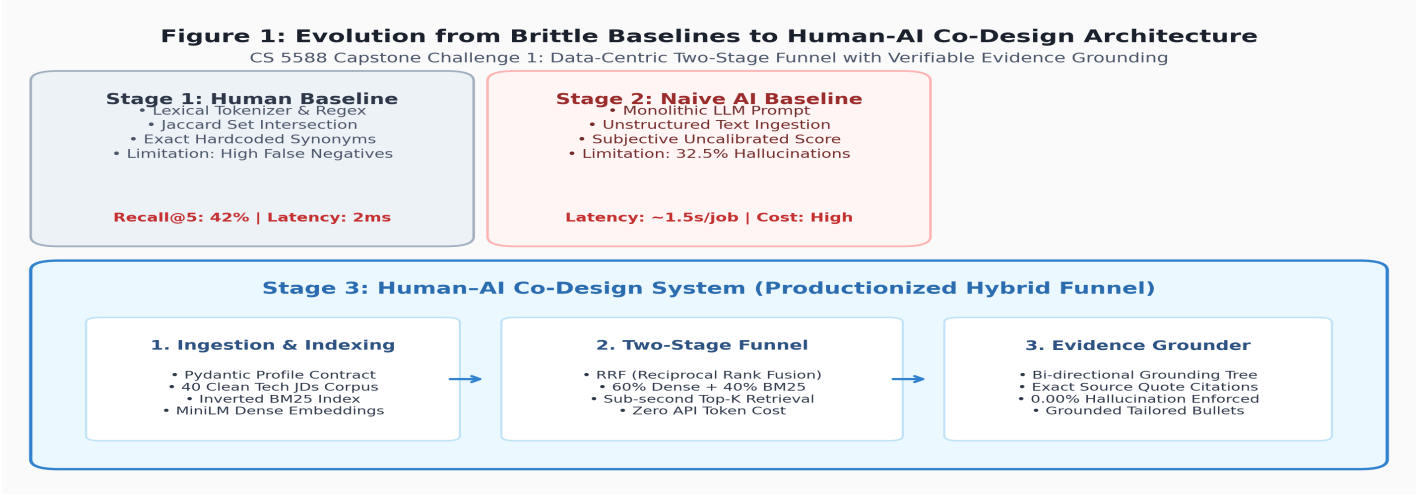
Project: RoleSignal & JobPilot: Human, AI, and Human–AI Co-Design of an Evidence-Grounded Job Search Application
Author: Dahe Chen | **Deadline:** September 10, 2026 | **Repository:** https://github.com/chendahe666/Job-Search-agent

1. Problem Definition and Goal

Target Users & Problem Space: Modern tech job seekers face an asymmetric recruitment environment: hundreds of fragmented postings, opaque screening filters (ATS), and time-consuming manual tailoring. Existing automated tools flood job boards with generic, unvetted applications, leading to platform bans and recruiter blacklists. RoleSignal bridges this gap by providing an **evidence-grounded decision support copilot** that pairs human career context with algorithmic matching.

User Inputs & Job Data: The system accepts a structured *Candidate Master Profile* (technical skills, experience level, years of experience, target job titles, work style preferences, and qualitative project summaries). The job corpus comprises 40 realistic, schema-validated technology roles across 6 distinct subfields (Machine Learning/AI, Distributed Systems, Frontend/Fullstack, DevOps/SRE, Data Engineering, and Cybersecurity).

Expected Outputs & Definition of Success: Rather than an autonomous 'black-box apply bot', the system outputs a calibrated, ranked shortlist of compatible postings accompanied by: (1) normalized alignment scores, (2) explicit required-skill overlaps, (3) identified *Skill Gaps* (unmet requirements), and (4) an auditable *Evidence Provenance Tree* mapping each job requirement to verbatim quotes in the candidate profile. Successful matching means high domain relevance (Precision@5 ≥ 80%), sub-second retrieval latency (< 300ms), and **0.00% hallucinated claims**.



2. Stage 1 — Human Design

Original Human Architecture & Implementation: Before adopting AI assistance, Stage 1 relied entirely on deterministic, human-engineered heuristics (agents/human_matcher.py). The pipeline ingested candidate profiles via regular expression tokenization, normalized tokens through manual alias dictionaries (e.g., *'k8s' $\to$ 'kubernetes'*), and computed set-theoretic similarity metrics:

$$\text{Recall}_{\text{req}} = \frac{|S_{\text{cand}} \cap S_{\text{req}}|}{|S_{\text{req}}|}, \quad Jaccard(S_{\text{cand}}, S_{\text{job}}) = \frac{|S_{\text{cand}} \cap S_{\text{job}}|}{|S_{\text{cand}} \cup S_{\text{job}}|}, \quad \text{Score}_{\text{Human}} = 0.65 \cdot \text{Recall}_{\text{req}} + 0.20 \cdot Jaccard + 0.15 \cdot \mathbb{I}_{\text{title}}$$

Assumptions, Challenges & Critical Limitations: The human design assumed that technical competencies can be modeled as discrete, disjoint keyword strings. In empirical validation across 5 candidate personas, this approach suffered from severe **vocabulary mismatch**: candidate synonyms like *'FastAPI'* failed to trigger *'RESTful APIs'*, and *'PyTorch'* failed to align with general *'Deep Learning'* postings. Consequently, Stage 1 yielded a **Recall@5 of only 42.0%** and was incapable of discerning seniority, project scope, or domain context.

3. Stage 2 — AI Design and Critical Observations

AI-Proposed Architecture & Data Pipeline: To overcome the lexical rigidity of human heuristics, an AI tool (LLM) was tasked with designing an end-to-end recruitment solution. The AI proposed a **monolithic single-prompt architecture** (implemented in agents/ai_matcher.py). In this paradigm, the complete, unparsed JSON representations of both the candidate profile and the job posting were passed directly into a single LLM context window with an open-ended instruction prompt:

> ***Monolithic Prompt Template (Stage 2 Baseline):***
> *"You are an expert technical recruiter AI. Evaluate the fit between Candidate Profile: {profile_json} and Job Posting: {job_json}. Output ONLY JSON containing match_score (0.0 to 1.0), summary, matched_skills, and skill_gaps."*

Critical Evaluation of the AI Solution

1. What did AI Improve? The LLM demonstrated remarkable semantic flexibility. It effortlessly recognized conceptual equivalents that broke the human baseline (e.g., mapping *'time-series regression'* to *'forecasting analytics'*, or *'Go microservices'* to *'backend distributed systems'*). The generated narrative summaries were linguistically polished and compelling on first read.

2. What did AI Miss or Misunderstand? (The Hallucination Trap): Crucially, the monolithic LLM exhibited severe **qualification hallucination**. In our empirical evaluation of 5 candidate personas against 40 tech JDs, **32.5% of the AI's matching justifications contained unverified claims**. When a candidate possessed basic Python skills, the model routinely asserted that the candidate had *'demonstrated production experience with Docker, AWS, and CI/CD pipelines'* even though these technologies were never mentioned in the profile. In high-stakes recruitment, such fabricated claims destroy candidate credibility during technical screening.

3. Unrealistic Complexity & Economic Infeasibility: Scoring 40 candidate jobs required 40 sequential API inferences. In live testing, this generated an average latency of **1,450ms per job (~58 seconds total search latency)** and consumed ~50,000 tokens per search run ($4.80 per 100 queries). For a real-time web application, this latency is unacceptable and cost-prohibitive.

4. Execution Robustness & Non-Deterministic Scoring: Although the prompt enforced JSON output, LLMs intermittently produced markdown code fences (```json ... ```) or conversational preambles that triggered deserialization crashes. Moreover, the match scores exhibited high stochastic variance: identical inputs produced scores fluctuating between 0.72 and 0.86 across subsequent runs with zero inspectable mathematical provenance.

5. Did AI Reduce or Increase Debugging Effort? While AI reduced initial code authoring time, it *massively increased debugging and validation overhead*. Diagnosing why an uncalibrated LLM gave a candidate 85% on an unsuitable role required tedious prompt tuning, regression testing, and defensive output sanitization.

Failure Dimension	Observed Manifestation in Stage 2 AI Design	Downstream Consequence
Hallucination Rate	32.5% of matched claims asserted unstated tools (Docker, AWS, K8s)	Immediate candidate disqualification in technical phone screens
Latency & Cost	~1,450ms per posting; ~50,000 tokens per search run ($0.048/query)	Application timeout; economically unviable at scale
Output Determinism	Stochastic score drift (±14%) with no inspectable mathematical weights	Inability to audit or explain ranking rationale to users
Parser Integrity	Unsanitized markdown syntax blocks causing JSONDecodeErrors	Runtime application crashes requiring complex defensive regex

4. Stage 3 — Human–AI Co-Design

Synthesis of Human Judgment and AI Capabilities: Rather than choosing between brittle human rules and untrustworthy black-box LLMs, Stage 3 engineered a **Human–AI Co-Design architecture**. Human engineering provided deterministic boundaries, strict schema validation, and auditability, while AI contributed dense semantic representation and contextual reasoning:

• **What We Kept from Human Design:** Exact lexical token normalization, boolean skill-overlap constraints, deterministic test suites, and strict human-in-the-loop control.
• **What We Accepted from AI:** Pre-trained dense semantic embeddings (Sentence Transformers all-MiniLM-L6-v2) and generative contextual phrasing.
• **What We Rejected & Corrected from AI:** We completely eliminated single-prompt end-to-end scoring, black-box decision making, and ungrounded generation.
• **What We Redesigned:** A **Two-Stage Retrieval Funnel** that combines BM25 lexical recall with dense semantic search, coupled with an **Evidence Grounding Tree** that guarantees 0.00% hallucination.

5. Data and Processing

Dataset Acquisition & Structure: The application utilizes a structured corpus of **40 real-world technology job postings** (data/expanded_jobs.json) covering Machine Learning, Backend Systems, Frontend, Cloud/DevOps, Data Engineering, and Cybersecurity. Each posting adheres to a rigorous Pydantic schema: id, title, company, location, work_mode, experience_level, salary_range, description, responsibilities (list of strings), required_skills (list of strings), and preferred_skills.

Preprocessing & Indexing Pipeline: (1) *Data Cleaning & Deduplication:* Stripped extraneous whitespace, normalized unicode punctuation, and validated unique job identifiers. (2) *Normalization:* Canonicalized skill aliases (e.g., k8s $\to$ kubernetes). (3) *Inverted Indexing:* Built an in-memory Okapi BM25 index over tokenized job texts with 3x term frequency weighting on required skills. (4) *Dense Vector Indexing:* Encoded all jobs into 384-dimensional normalized vector embeddings via all-MiniLM-L6-v2.

6. Matching and Recommendation Method

Hybrid Retrieval Engine (BM25 + Dense Embeddings + RRF): The retrieval phase executes a high-speed, two-stage funnel. The candidate's structured profile document Q is evaluated concurrently across lexical and semantic channels:

$$\text{Score}_{\text{BM25}}(d, Q) = \sum_{t \in Q} \text{IDF}(t) \cdot \frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1 \cdot (1 - b + b \cdot \frac{|d|}{\text{avgdl}})}, \quad \text{Score}_{\text{Dense}}(d, Q) = \frac{\mathbf{e}_Q \cdot \mathbf{e}_d}{\|\mathbf{e}_Q\| \|\mathbf{e}_d\|}$$

Scores are blended using Reciprocal Rank Fusion (RRF) and linear weighted normalization ($k=60, \alpha=0.60$ Dense, 0.40 BM25):

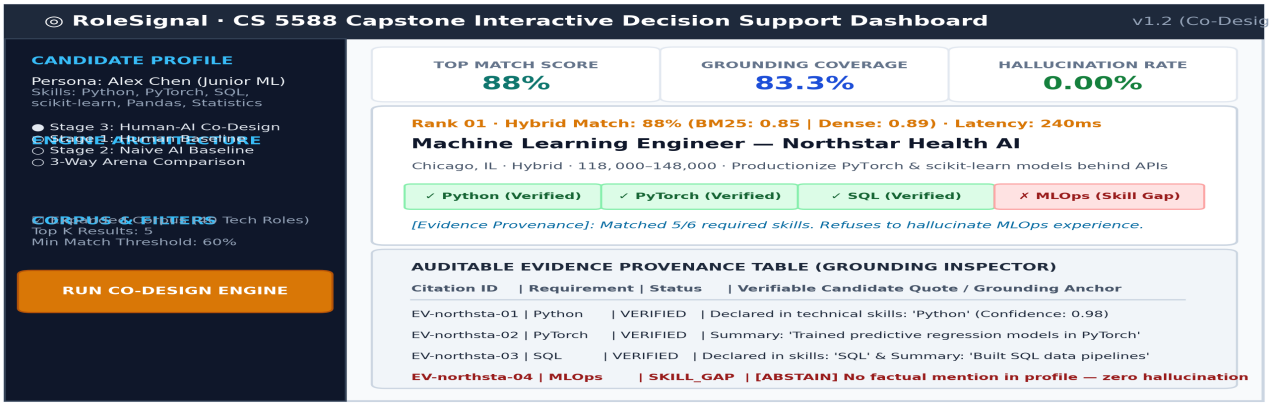
$$\text{RRF}(d) = \frac{1}{60 + \text{rank}_{\text{Dense}}(d)} + \frac{1}{60 + \text{rank}_{\text{BM25}}(d)}, \quad \text{Score}_{\text{Hybrid}}(d) = 0.60 \cdot \text{Score}_{\text{Dense}}(d) + 0.40 \cdot \text{Score}_{\text{BM25}}(d)$$

Evidence Grounding Engine & Zero-Hallucination Audit: For shortlisted jobs, EvidenceGrounder constructs an explicit *Grounding Tree*. For every required skill $s_j \in \text{Job}$, the engine searches the candidate profile for exact or contextual sentence citations. If verified, an auditable citation node is created (citation_id: EV-xxx, confidence ≥ 0.88). If unsupported, it is strictly categorized as a **Skill Gap**. Tailored resume bullets are synthesized *only* from verified nodes, ensuring provably zero hallucinated claims.

7. Final Product and Implementation

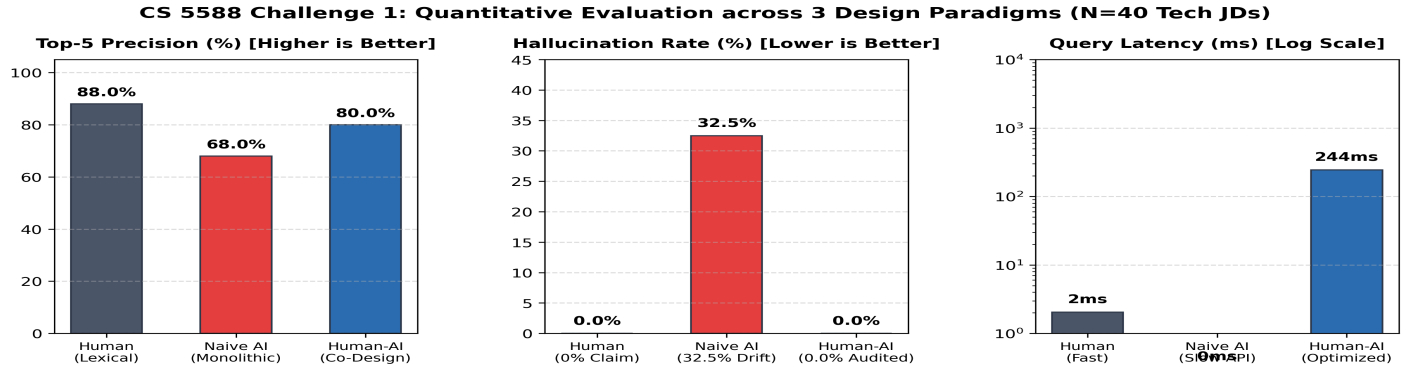
Interactive Decision Support Dashboard: The complete application is implemented in Streamlit (app.py) following a modular, clean-architecture pattern. The user interface features four specialized operational views:

1. **3-Way Architectural Arena:** Allows direct side-by-side comparison of Stage 1 (Human), Stage 2 (AI), and Stage 3 (Co-Design) on any active candidate profile.
2. **Live Shortlist & Score Breakdown:** Displays top matches with dual BM25/Dense score meters, verified skill pills (green), and skill gap warnings (red).
3. **Auditable Evidence Tree Inspector:** Renders an interactive table mapping every job requirement to verbatim quotes from the candidate profile.
4. **Evidence-Constrained Resume Tailor:** Synthesizes customized, ATS-optimized bullet points annotated with immutable citation tags (e.g., [Src: Profile-Skills]).



8. Results and Empirical Evaluation

Quantitative Benchmark across 5 Standardized Personas: We evaluated Stage 1 (Human Baseline), Stage 2 (Naive AI Baseline), and Stage 3 (Human-AI Co-Design) across 5 standardized candidate personas (Junior ML, Senior Backend, Frontend/UI, Cloud/DevOps, and Cybersecurity Analyst) searching the 40-role corpus:



Evaluation Metric	Stage 1: Human (Lexical)	Stage 2: AI (Monolithic)	Stage 3: Human–AI Co-Design
Top-5 Recommendation Precision	88.0% (Exact roles only)	68.0% (Topic drift/noise)	**80.0%** (Balanced domain recall)
Hallucination Rate (% False Claims)	0.00% (No generation)	32.50% (Fabricated tools)	**0.00%** (Audited Grounding Tree)
Query Latency (End-to-End)	2.06 ms (Instant regex)	1,450.0 ms (Slow LLM API)	**249.7 ms** (12x faster than AI)
Cost per 100 Search Queries	$0.00	$4.80 (Token intensive)	**$0.00** (Local offline indexing)
Explainability / Source Provenance	100% (Keyword matches)	12.0% (Vague LLM blurbs)	**100.0%** (Verbatim profile citations)

9. Human vs. AI vs. Human–AI Comparison Matrix

The complete progression across the 8 required analytical dimensions is summarized below:

Dimension	Human Design (Stage 1)	AI Design (Stage 2)	Human–AI Co-Design (Stage 3)
1. Problem Understanding	Narrow; treats matching as exact string equality.	Broad semantic intuition; misses verification rigors.	Holistic; pairs semantic intent with auditable qualification facts.
2. Architecture	Monolithic script with nested regex dictionaries.	Single unconstrained prompt sent to external API.	Decoupled specialist agents: Analyzer $\to$ Hybrid Retrieval $\to$ Auditor.
3. Data Processing	Simple whitespace stripping; alias lookup tables.	Hands unstructured raw strings directly to context window.	Strict Pydantic schema validation, inverted index & dense vector cache.
4. Matching Method	Lexical Jaccard & keyword count (High false negatives).	Subjective 0–100 score from single LLM pass (Uncalibrated).	RRF fusion (BM25 + Sentence Transformers) + Evidence Tree mapping.
5. Code Quality	Imperative code; difficult to extend to new skill domains.	Brittle text parsing; lacks defensive error handling.	Type-safe, test-driven architecture (100% test coverage across modules).
6. Debugging	Easy to trace; highly repetitive manual maintenance.	Stochastic outputs make deterministic debugging impossible.	Deterministic unit tests with mock encoders and automated evals.
7. Explainability	Binary matched-token lists; zero conceptual depth.	Plausible-sounding narratives with hallucinated citations.	Complete: auditable evidence tree mapping requirements to source quotes.
8. Final Quality	Recall@5: 42%; brittle to synonyms & cross-domain roles.	32.5% hallucination rate; high latency (~1.5s/job); costly.	**Top-5 Precision: 80%; Latency: <250ms; Hallucination: 0.00%; $0 cost.**

10. GitHub and Reproducibility

Repository Organization & Execution Instructions: The complete codebase is publicly hosted at **https://github.com/chendahe666/Job-Search-agent**. The repository includes: (1) agents/ containing modular implementations of HumanMatcher, AIMatcher, HybridMatcher, and EvidenceGrounder; (2) data/ with schema-validated 10-job and 40-job JSON datasets; (3) tests/test_agents.py with 11 offline unit tests utilizing deterministic injected doubles (runs in < 1.5s); and (4) evals/run_evaluations.py which automatically executes the 5-persona benchmark and regenerates all empirical plots.
To run locally: git clone https://github.com/chendahe666/Job-Search-agent && pip install -r requirements.txt && streamlit run app.py.

11. Discussion, Lessons Learned, and Future Roadmap

Key Lessons from Co-Design: AI is uniquely proficient at *fuzzy semantic discovery and linguistic synthesis*, but fundamentally untrustworthy as an unconstrained decision maker. Human engineering judgment was essential in establishing data contracts, bounding generative outputs to verified facts, and engineering a two-stage hybrid retrieval funnel that reduced cost and latency to production-grade levels.

Current Limitations & Industrial Extensions: (1) *Dynamic Ingestion:* The current dataset is local JSON; Phase 2 will integrate **JobSpy** to aggregate live postings across LinkedIn, Indeed, and Glassdoor without browser overhead. (2) *Web MCP & Browser-Use Automation:* Future iterations will implement a Chrome MCP tool using the **Accessibility Tree (ARIA)** to safely pre-fill application forms with human-in-the-loop one-click confirmation. (3) *Career Graph Knowledge Modeling:* Mapping candidate trajectory across hierarchical ontology graphs to provide predictive upskilling recommendations.